1

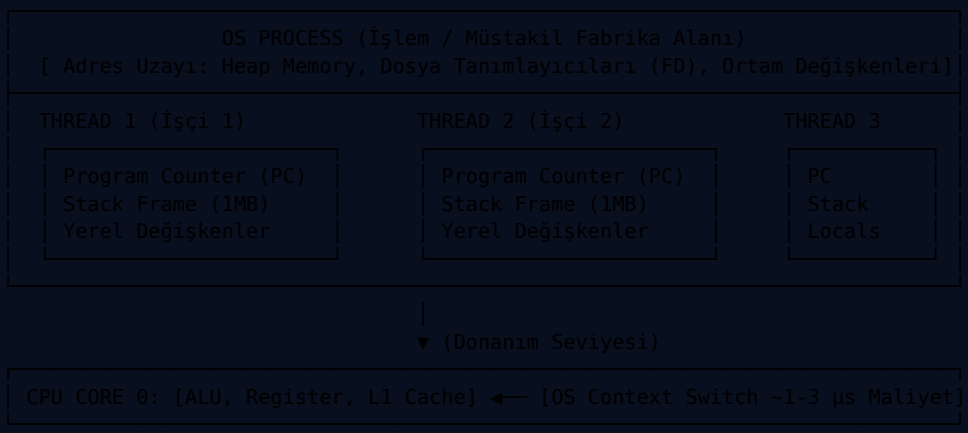
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Nedir? Process vs Thread, OS Context Switching & CPU Çekirdekleri

What is a Thread? Process vs Thread, OS Context Switching & CPU Cores

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir Process (İşlem), etrafı yüksek duvarlarla çevrili koca bir fabrikadır. Kendi su tesisatı, elektrik sayacı ve tapusu (ayrı RAM adres alanı) vardır. Bir fabrikanın diğer fabrikanın odalarına izinsiz girmesi yasaktır! Thread (İş parçacığı) ise o fabrikanın içinde koşturan işçilerdir. Bütün işçiler aynı fabrikanın ortak deposunu (Heap Memory) paylaşırlar; ama her işçinin cebinde kendi özel not defteri (Thread Stack) vardır. İki işçi aynı depodaki aynı kutuya (Paylaşılan Değişken) haberleşmeden aynı anda dalarsa ortalık karışır (Race Condition)!

🇬🇧 ENGLISH (ELI5)

A Process is an entire secure factory compound with its own private land, utilities, and security gate (isolated virtual address space). Process A cannot inspect Process B's floor plan. A Thread is a worker inside that factory. All workers walk the same shared factory warehouse floor (Heap Memory), but each worker carries a personal pocket notepad (Thread Stack). If two workers update the same inventory box simultaneously without coordination, inventory records corrupt instantly (Race Condition)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG seviyesinde Process vs Thread anatomisi: 1) Bellek İzolasyonu: Process'ler MMU (Memory Management Unit) ve Page Table ile sanal adres alanlarında tamamen izole edilir. İletişim IPC (Inter-Process Communication: Sockets, Shared Memory, Pipes) gerektirir ve pahalıdır. 2) Thread Paylaşımı: Bir JVM process'indeki tüm platform thread'leri Heap, Metaspace ve dosya tanıtıcılarını (FD) paylaşır; ancak her thread kendi Stack alanına (1MB varsayılan), Program Counter (PC) register'ına ve Native Method Stack'ine sahiptir. 3) Context Switch Maliyeti: İşletim sistemi çekirdeği (Kernel) iki platform thread'i arasında geçiş yaparken CPU register'larını kaydeder, TLB (Translation Lookaside Buffer) önbelleğini temizler ve L1/L2 CPU cache'lerini bozar (Cache Pollution). Bu maliyet 1-3 mikrosaniyedir ve saniyede milyonlarca istek alan MAANG sistemlerinde verim kaybına yol açar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG-level architectural mechanics: 1) Address Space Isolation: Processes operate in distinct virtual memory boundaries governed by the hardware MMU and OS page tables. Cross-process IPC requires context-switching into ring-0 kernel space. 2) Intra-Process Thread Concurrency: Java platform threads map 1:1 to OS native threads (POSIX pthreads / Windows kernel threads). They share the JVM Heap and Metaspace, but retain private Program Counters and runtime call stacks (1MB default). 3) Context Switching Overhead: When the OS scheduler preempts Thread A for Thread B, it saves instruction pointers and registers to kernel memory, often triggering TLB invalidation and L1/L2 cache misses (Cache Pollution), incurring 1-3 microseconds of pure CPU latency per switch.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Platform Thread vs Donanım Çekirdekleri Bilgisi
void main() {
    int availableCores = Runtime.getRuntime().availableProcessors();
    long totalMemoryMb = Runtime.getRuntime().totalMemory() / (1024 * 1024);

    IO.println("CPU Donanım Çekirdek Sayısı: " + availableCores);
    IO.println("JVM Tahsis Edilen Heap Bellek: " + totalMemoryMb + " MB");

    // Geleneksel Platform Thread başlatma
    Thread osThread = new Thread(() -> {
        var current = Thread.currentThread();
        IO.println("Çalışan İşçi: " + current.getName() + " | Platform Thread mi: " + !
current.isVirtual());
    }, "MAANG-Worker-01");

    osThread.start();
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Daha fazla thread açarsak sistemimiz her zaman daha mı hızlanır?' diye sorar. Yanıt: Hayır! CPU çekirdek sayısından çok fazla platform thread açılırsa, CPU zamanını iş yapmaya değil işçiler arasında geçiş yapmaya (Context Switching thrashing) harcar ve throughput çöker!

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: The interviewer asks: 'Can we endlessly boost throughput simply by adding more threads?' Senior Answer: 'No! If runnable platform thread count drastically exceeds physical CPU cores, the kernel spends more clock cycles on thread context switching and cache invalidation than actual task computation, precipitating throughput collapse.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Process'ler izole adres alanlarına sahiptir; Thread'ler ise aynı Heap alanını paylaşır.

• Platform thread'leri 1:1 işletim sistemi kernel thread'lerine eşlenir (~1MB stack).

• Gereğinden fazla thread context-switching thrashing'e yol açarak sistemi kilitler.

🎯 Key Takeaways for the Interview

• Processes own isolated virtual address spaces; threads share the JVM heap.

• Platform threads map 1:1 to kernel threads, incurring ~1MB memory footprint.

• Oversubscribing threads degrades throughput via context-switch thrashing.